

Laporan Praktikum Kontrol Cerdas

Nama : Farhat Fadel Yusmanan

NIM : 224308033

Kelas : TKA-6B

Akun Github (Tautan) : <https://github.com/farhatfadel17>

Student Lab Assistant :Mbak Dwi

1. Judul Percobaan : ***Reinforcement Learning for Autonomous Control***

2. Tujuan Percobaan :

- Memahami konsep dasar Reinforcement Learning (RL) dalam sistem kendali.
- Mengimplemtasikan agen RL menggunakan algoritma *Deep Q-Network* (DQN).
- Menggunakan OpenAI Gym sebagai simulasi lingkungan untuk pelatihan RL.
- Melatih dan menguji agen RL untuk mengontrol lingkungan secara otonom.
- Menggunaka GitHub untuk version control dan dokumentasi praktikum.

3. Landasan Teori :

Machine Learning merupakan sistem yang mampu belajar sendiri untuk memutuskan sesuatu tanpa harus berulang kali diprogram oleh manusia (*Karimah Tauhid, Volume 2 Nomor 1 (2023), e-ISSN 2963-590X, 2023*). Untuk bisa menggunakan *machine learning* harus memiliki data. Data yang digunakan dibagi menjadi dua data training dan data testing. Data training digunakan untuk melatih algoritma, sedangkan data testing digunakan untuk mengetahui performa yang telah dilatih sebelumnya ketika menemukan data baru yang belum pernah diberikan dalam data training (Yudianto & Fatta, 2020). Machine Learning memiliki beberapa jenis, seperti, supervised learning, unsupervised learning dan reinforcement learning.

Reinforcement Learning adalah proses mapping (pemetaan) dari situasi yang ada di environment (states) ke bentuk aksi (behavior) agar dapat memaksimalkan reward. Agent yang bertindak sebagai sang learner tidak perlu diberitahukan

behavior apakah yang akan sepatutnya dilakukan, atau dengan kata lain, sang learner belajar sendiri dari pengalamannya.(Angelina et al., n.d.).

Deep Q-Network (DQN) adalah algoritma dalam reinforcement learning yang menggabungkan Q-learning dengan jaringan saraf dalam untuk menyelesaikan masalah kompleks, sehingga memungkinkan agen untuk belajar dari pengalaman dan membuat keputusan dalam lingkungan yang rumit. Algoritma ini sangat berguna untuk menangani masalah dengan ruang keadaan yang besar, seperti pengenalan gambar dan kontrol robot, dengan memanfaatkan kemampuan jaringan saraf untuk mengekstrak fitur dari data secara efisien dan efektif. DQN bekerja dengan cara memanfaatkan jaringan saraf untuk memperkirakan nilai Q, yang merupakan nilai dari setiap aksi yang mungkin diambil dalam suatu keadaan tertentu. Dengan menggunakan pemutaran ulang pengalaman, DQN menyimpan pengalaman masa lalu dalam memori dan mengambil sampel acak dari pengalaman tersebut untuk memperbarui model, sehingga meningkatkan stabilitas dan efisiensi pembelajaran. Selain itu, DQN juga menerapkan strategi eksplorasi epsilon-greedy, yang memungkinkan agen untuk menyeimbangkan antara eksplorasi tindakan baru dan eksploitasi tindakan yang sudah diketahui, sehingga agen dapat menemukan strategi yang optimal seiring waktu(Baradja & Tjendrowasono, 2024).

4. Analisis dan Diskusi :

-Analisis

MountainCar-v0 adalah lingkungan di mana sebuah mobil harus mencapai puncak bukit meskipun tidak memiliki daya yang cukup untuk langsung mencapainya. Lingkungan ini memiliki ruang keadaan dua dimensi, yang terdiri dari posisi mobil di sepanjang bukit dan kecepatan mobil. Terdapat tiga aksi yang dapat diambil oleh agen, yaitu bergerak ke kiri, tetap diam, atau bergerak ke kanan. Agen harus memilih tindakan yang tepat untuk mengoptimalkan pergerakan mobil. Dalam lingkungan ini, agen menerima reward standar sebesar -1 untuk setiap langkah yang diambil, yang mendorong agen untuk mencapai puncak bukit secepat mungkin. Namun, dalam beberapa implementasi, reward ini dapat dimodifikasi untuk mempercepat proses pembelajaran. Tantangan utama adalah bahwa mobil harus

mengayun bolak-balik untuk mendapatkan momentum yang cukup agar dapat mencapai puncak bukit. Reinforcement Learning (RL) digunakan untuk melatih agen dalam lingkungan ini. Agen belajar melalui interaksi dengan lingkungan, mencoba berbagai tindakan, dan menerima umpan balik dalam bentuk reward atau punishment. Dengan menggunakan algoritma RL, agen dapat mengembangkan kebijakan yang optimal untuk mencapai tujuan, belajar dari pengalaman sebelumnya, dan beradaptasi dengan kondisi yang berubah. Karena sifatnya yang kontinu. Penggunaan Deep Q-Network (DQN) menjadi pendekatan yang baik, karena DQN dapat menangani ruang keadaan yang besar dan kompleks dengan menggunakan jaringan saraf untuk memperkirakan nilai Q, sehingga memungkinkan agen untuk belajar dengan lebih efisien dalam mencapai puncak bukit.

-Diskusi

Deep Q-Network (DQN) pada lingkungan MountainCar-v0 bertujuan untuk melatih agen agar dapat mencapai puncak bukit dengan menggunakan aksi yang optimal. Lingkungan ini menantang karena mobil harus mengumpulkan momentum dengan cara mengayun bolak-balik sebelum dapat mencapai tujuan. Model DQN yang digunakan memiliki struktur sederhana dengan tiga lapisan. Lapisan input terdiri dari 24 neuron dengan aktivasi ReLU, lapisan hidden juga memiliki 24 neuron dengan aktivasi ReLU, dan lapisan output terdiri dari 3 neuron yang mewakili jumlah aksi dengan aktivasi linear. Tantangan utama yang dihadapi dalam implementasi DQN ini adalah waktu yang diperlukan oleh agen untuk menemukan strategi optimal. Pada fase awal pelatihan, agen sering kali melakukan aksi acak, yang dapat memperlambat proses pembelajaran.

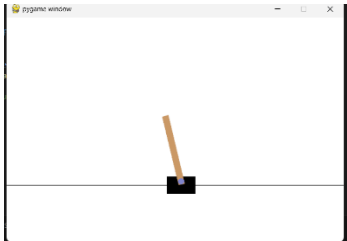
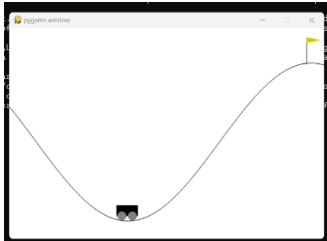
5. Assignment :

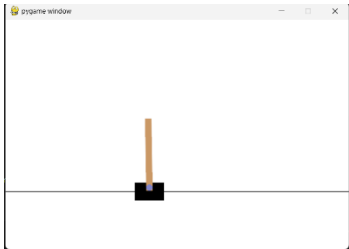
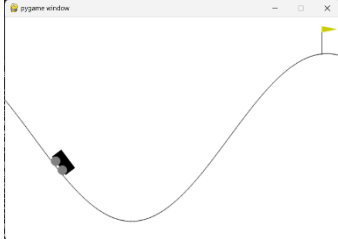
Tugas percobaan praktikum pada minggu keempat ini yaitu implementasi dari algoritma Deep Q-Network (DQN) yang digunakan untuk melatih environment agen dalam environment MountainCar-v0. Tujuan dari agen ini yaitu mencapai puncak bukit dengan menggunakan aksi yang optimal. Kode dimulai dengan mengimpor beberapa library yang diperlukan, seperti gymnasium untuk

environment reinforcement learning, numpy untuk operasi numerik, random untuk pemilihan acak, dan deque dari collections untuk menyimpan pengalaman. Selain itu, tensorflow.keras digunakan untuk membangun model jaringan saraf. Selanjutnya, terdapat kelas yang disebut DQNAgent, yang mendefinisikan agen DQN. Agen diinisialisasi dengan ukuran keadaan dan ukuran aksi. Beberapa parameter penting juga digunakan, seperti gamma yang berfungsi sebagai faktor diskon, epsilon yang merupakan parameter untuk eksplorasi, dan learning_rate yang digunakan untuk optimasi model.

Pada eksperimen pertama **CartPole-V1**, agen harus menjaga tiang tetap tegak di atas gerobak/kotak. Lingkungan ini cukup sederhana, sehingga agen dapat belajar dengan cepat. Setiap kali agen berhasil menjaga tiang tetap tegak, ia mendapatkan reward positif. Pelatihan untuk CartPole bisa dilakukan dalam waktu singkat, hanya dalam beberapa ratus episode. Agen dapat dengan stabil mencapai tujuannya karena lingkungan ini lebih mudah diprediksi. Sedangkan, pada **MountainCar-v0**, agen harus menggerakkan mobil ke puncak bukit, tetapi mobil tidak memiliki cukup tenaga untuk langsung mencapai tujuan. Agen harus belajar membangun momentum dengan bergerak maju dan mundur. Reward di **MountainCar-v0** lebih susah karena mobil tidak berhasil mencapai puncak, yang membuat pembelajaran menjadi lebih sulit. Model yang digunakan memerlukan arsitektur yang lebih kompleks atau teknik tambahan untuk meningkatkan stabilitas. Proses pelatihan untuk MountainCar memerlukan lebih banyak episode. sebelum agen dapat mencapai kinerja yang baik. Pembaruan nilai dalam lingkungan ini bisa lebih berfluktuasi, sehingga kembali ke skor yang lebih tinggi.

6. Data dan Output Hasil :

No.	Variabel	Hasil Pengamatan	
		Chart pole v1	Mountain Car v0
1.	Waktu Pelatihan	Chartpole lebih cepat dalam mencapai performa optimal saat episode diatas 100	Mountain car membutuhkan waktu lama untuk mencapai performa yang optimal
2.	Kinerja Agen	Kinerja yang optimal dengan skor yang lebih tinggi	Masih menghadapi tantangan untuk mencapai puncak
3.	Kondisi Berhasil	Episode berakhir jika tiang jatuh atau batas langkah maksimum tercapai	Episode berakhir jika mobil mencapai puncak bukit (flag)
4.	Hasil Episode Awal	<pre>Episode: 1/1000, Score: 29, Epsilon: 1.00 Episode: 2/1000, Score: 31, Epsilon: 1.00 Episode: 3/1000, Score: 15, Epsilon: 0.99 Episode: 4/1000, Score: 16, Epsilon: 0.99 Episode: 5/1000, Score: 28, Epsilon: 0.99 Episode: 6/1000, Score: 12, Epsilon: 0.98</pre> 	<pre>2025-09-09 22:26:57.535653: I tensorflow/core/p use available CPU instructions in performance-c To enable the following instructions: AVX2 FMA, ags. Episode: 1/1000, Score: 199, Epsilon: 1.00 Episode: 2/1000, Score: 199, Epsilon: 0.99 Episode: 3/1000, Score: 199, Epsilon: 0.99 Episode: 4/1000, Score: 199, Epsilon: 0.99</pre> 

5.	Hasil Episode Akhir	<pre> Episode: 258/1000, Score: 32, Epsilon: 0.29 Episode: 254/1000, Score: 9, Epsilon: 0.28 Episode: 267/1000, Score: 23, Epsilon: 0.26 Episode: 287/1000, Score: 61, Epsilon: 0.24 Episode: 288/1000, Score: 98, Epsilon: 0.24 Episode: 302/1000, Score: 64, Epsilon: 0.22 Episode: 307/1000, Score: 64, Epsilon: 0.22 Episode: 332/1000, Score: 70, Epsilon: 0.19 Episode: 334/1000, Score: 25, Epsilon: 0.19 </pre> 	<pre> Episode: 991/1000, Score: -200, Epsilon: 0.01 Episode: 992/1000, Score: -200, Epsilon: 0.01 Episode: 993/1000, Score: -200, Epsilon: 0.01 Episode: 994/1000, Score: -200, Epsilon: 0.01 Episode: 995/1000, Score: -200, Epsilon: 0.01 Episode: 996/1000, Score: -200, Epsilon: 0.01 Episode: 997/1000, Score: -200, Epsilon: 0.01 Episode: 998/1000, Score: -200, Epsilon: 0.01 Episode: 999/1000, Score: -200, Epsilon: 0.01 Episode: 1000/1000, Score: -200, Epsilon: 0.01 </pre> 
----	---------------------	---	---

7. Kesimpulan

Berdasarkan praktikum dan analisis yang telah dilakukan, maka dapat diambil kesimpulan yaitu:

- CartPole-v1 lebih cepat dipelajari karena memberikan *reward* di setiap langkah, sehingga agen dapat dengan cepat memahami strategi optimal.
- MountainCar-v0 lebih sulit dipelajari karena *reward* hanya diberikan saat mencapai puncak bukit, memerlukan strategi eksplorasi lebih lama, dan mengandalkan momentum untuk berhasil.
- Penggunaan target *network* membantu stabilitas pembelajaran, menghindari perubahan nilai Q yang terlalu drastis.
- Ukuran batch atau batch size akan mempengaruhi hasil dari pelatihan

8. Saran :

Untuk meningkatkan agar performa agen DQN lebih baik bisa menerapkan *Target Network* untuk meningkatkan stabilitas dan mencegah model dari *overfitting* terhadap pola jangka pendek, dengan cara memperbarui target *network* secara periodik. strategi epsilon *decay* yang lebih adaptif, seperti menyesuaikannya berdasarkan performa, dapat memperbaiki keseimbangan eksplorasi dan eksploitasi. dapat mengoptimalkan *hyperparameter* seperti gamma, learning_rate, dan

epsilon_decay untuk meningkatkan performa. Menggunakan Double DQN akan membantu menghindari masalah overestimation. MountainCar, pendekatan lain seperti *actor-critic methods* atau *policy-based reinforcement learning* dapat dicoba untuk meningkatkan efisiensi pembelajaran.

9. Daftar Pustaka :

- Angelina, L., Purwanto, Y., & Novianty, A. (n.d.). *Pengambilan Keputusan Pada Trafik Management Dengan Menggunakan Reinforcement Learning Decision Making on Traffic Management Using Reinforcement Learning*.
- Baradja, A., & Tjendrowasono, T. I. (2024). Pengaplikasian Deep Reinforcement Q-Learning Untuk Prediksi Perdagangan Valas Otomatis. *Jurnal Rekayasa Sistem Informasi dan Teknologi*, 1(3), 190–198. <https://doi.org/10.59407/jrsit.v1i3.519>
- Karimah Tauhid, Volume 2 Nomor 1 (2023), e-ISSN 2963-590X. (2023). 2.
- Yudianto, M. R. A., & Fatta, H. A. (2020). *ANALISIS PENGARUH TINGKAT AKURASI KLASIFIKASI CITRA WAYANG DENGAN ALGORITMA CONVOLUTIONAL NEURAL NETWORK*.